

COMP0086: Bayesian Deep Learning - Lecture Notes

Comprehensive Course Notes

Contents

1	Part I: Foundational Optimization and Learning	5
1.1	Regularization and Bayesian Priors	5
1.2	Loss Functions for Classification	5
1.3	Gradient Descent and Stochastic Optimization	5
1.4	Momentum and Adaptive Methods	6
1.5	Quasi-Newton Methods	6
2	Part II: Automatic Differentiation and Computation	7
2.1	Computing Gradients Efficiently	7
2.2	Dual Numbers and Forward Mode	7
2.3	Handling Constraints via Transformations	7
2.4	Minibatching with Priors	7
3	Part III: Bayesian Inference and Posterior Approximation	8
3.1	Posterior Approximation with Laplace	8
3.2	Variational Inference and ELBO	8
3.3	Reparametrization Trick	8
3.4	Understanding KL Divergence Modes	8
3.5	Sequential Learning	9
4	Part IV: Deep Learning Architectures	10
4.1	Deep Feature Learning	10
4.2	Multi-layer Perceptrons	10
4.3	Convolutional Neural Networks	10
4.4	Network Depth and Architecture Improvements	11
4.5	Bayesian Approaches to Deep Models	11
5	Part V: Uncertainty Quantification and Probabilistic Models	12
5.1	Uncertainty in Neural Networks	12
5.2	Laplace Approximation for Deep Networks	12
5.3	Model Calibration	12
5.4	Epistemic vs. Aleatoric Uncertainty	13
5.5	Variational Autoencoders	13
5.6	Semi-Supervised Learning with VAEs	13
5.7	Neural Networks as Gaussian Processes	14
5.8	Information Theory Basics	14

6	Part VI: Advanced Applications and Inference Methods	15
6.1	Active Learning	15
6.2	Bayesian Active Learning by Disagreement (BALD)	15
6.3	Bayesian Optimization	15
6.4	Acquisition Functions	15
6.5	Langevin Dynamics for Posterior Sampling	16
6.6	Stochastic Gradient Langevin Dynamics (SGLD)	16
6.7	Wake-Sleep Algorithm	16

Introduction

This course covers the intersection of Bayesian inference and deep learning, exploring uncertainty quantification, efficient computation, and the connection between probabilistic inference and neural network learning.

1 Part I: Foundational Optimization and Learning

1.1 Regularization and Bayesian Priors

A regularizer $r(w)$ of order p is:

$$r(w) = \sum_i |w_i|^p, \quad p > 0$$

The L^2 regularization (Ridge) corresponds to a Gaussian prior. The L^1 regularization (Lasso) corresponds to a Laplace prior. The Laplace distribution is:

$$p(w) = \frac{1}{2b} \exp\left(-\frac{|w - \mu|}{b}\right)$$

1.2 Loss Functions for Classification

For binary classification:

$$\sum_i y_i \log f(x_i, w) + (1 - y_i) \log(1 - f(x_i, w))$$

For multi-class classification with softmax:

$$\hat{y}_{ik} = \text{softmax}(z_i) = \frac{\exp(z_{ik})}{\sum_j \exp(z_{ij})}$$

The loss function is:

$$\ell(z_i, y_i) = - \sum_{j=1}^K y_{ij} \log \hat{y}_{ij}$$

1.3 Gradient Descent and Stochastic Optimization

The full loss is:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i, \theta), y_i)$$

Stochastic gradient descent estimates using mini-batches:

$$\nabla_{\theta} L(\theta) \approx \frac{1}{m} \sum_{j=1}^m \nabla_{\theta} \ell(f(x_j[\theta], y_j))$$

Small batches lead to faster updates; large batches provide better gradient estimates.

Moving Average: The moving average at step t is updated as:

$$\bar{a}_t = \frac{1}{t} \sum_{k=1}^t x_k = \epsilon_t x_t + \mu_t \bar{a}_{t-1}$$

where if μ_t is small, more recent samples contribute more strongly.

1.4 Momentum and Adaptive Methods

Momentum-based updates accelerate convergence by accumulating gradients:

$$g_{t+1} = \mu g_t - \nabla f(\theta_t), \quad \theta_{t+1} = \theta_t + g_{t+1}$$

AdaGrad adapts learning rates per parameter using accumulated squared gradients:

$$\nabla_i = \nabla f(\theta)_i, \quad G_{t,ii}^t = \sum_{k=1}^t (\nabla_i)^2, \quad \theta_t^m = \theta_{t-1}^m - \frac{\eta}{\sqrt{G_{t,tt}^m + \epsilon}} g_t^m$$

However, accumulated gradients only increase, causing learning rates to decay to zero.

RMSprop uses a moving average of squared gradients to solve this:

$$S_{t,i} = \gamma S_{t-1,i} + (1 - \gamma) g_{t,i}^2, \quad \theta_t^m = \theta_t^m - \frac{\sqrt{S_{t-1,i} + \epsilon}}{g_{t,i}} g_{t,i}$$

Adam Optimizer combines RMSprop with Momentum and adds bias correction:

$$m_{t,i} = \beta_1 m_{t-1,i} + (1 - \beta_1) g_{t,i}, \quad \hat{m}_{t,i} = \frac{m_{t,i}}{1 - \beta_1^t}$$

$$s_{t,i} = \beta_2 s_{t-1,i} + (1 - \beta_2) g_{t,i}^2, \quad \hat{s}_{t,i} = \frac{s_{t,i}}{1 - \beta_2^t}$$

$$\theta_t^m = \theta_t^m - \frac{\eta}{\sqrt{\hat{s}_{t,i} + \epsilon}} \hat{m}_{t,i}$$

1.5 Quasi-Newton Methods

Higher-order optimization methods find B satisfying the secant condition:

$$\nabla f(\theta_{t+1}) = \nabla f(\theta_c) + B(\theta_{t+1} - \theta_c)$$

These achieve superlinear convergence order.

2 Part II: Automatic Differentiation and Computation

2.1 Computing Gradients Efficiently

Automatic differentiation computes gradients via the chain rule. There are two principal modes:

- **Forward mode:** Computes Jacobian-vector products. Memory-efficient to implement but best when the number of inputs is much smaller than outputs.
- **Backward mode:** Computes vector-Jacobian products. More memory-intensive but efficient when inputs $\gg$ outputs (common in deep learning).

Forward propagation: $\dot{u}_{t+1} = \frac{\partial u_{t+1}}{\partial u_t} \dot{u}_t$

Backward propagation: $\bar{V}_n = D_n \bar{V}_{n+1}$

2.2 Dual Numbers and Forward Mode

Dual numbers extend the reals with an infinitesimal ϵ where $\epsilon^2 = 0$:

$$f(x + \epsilon) = f(x) + f'(x)\epsilon$$

Forward-mode automatic differentiation uses dual number arithmetic to compute derivatives while evaluating the function.

2.3 Handling Constraints via Transformations

For constrained parameters (e.g., positive variances), use unconstrained representations:

$$\sigma = \exp(s), \quad s \in \mathbb{R}$$

For structured covariance matrices, use Cholesky or similar factorizations:

$$\Sigma = AA^T + \text{diag}(e^s)$$

This reduces parameters from $O(D^2)$ to $O(Dr + D)$ for rank r .

2.4 Minibatching with Priors

When using minibatches, scale the prior term to match the dataset size:

$$L(w) = \log p(w) + \sum_{i=1}^n \log p(z_i|w, x_i)$$

For minibatch M of size N/M :

$$L_M(w) = \log p(w) + \frac{N}{M} \sum_{i=1}^M \log p(y_i|w, x_i)$$

This ensures unbiased gradient estimates for the full objective.

3 Part III: Bayesian Inference and Posterior Approximation

3.1 Posterior Approximation with Laplace

The Laplace approximation approximates the posterior with a Gaussian distribution centered at the MAP estimate:

$$\log p(w|D) \approx \log \mathcal{N}(w|w^*, \Lambda^{-1}) = -\frac{1}{2}(w - w^*)^T \Lambda (w - w^*) + \text{const}$$

where $w^* = w_{\text{map}}$ and $\Lambda = -\nabla^2 \log p(w|D)$ is the Hessian at the MAP.

The marginal likelihood with Laplace approximation is:

$$p(D) \approx p(D|\theta^*)p(\theta^*) \frac{(2\pi)^{D/2}}{|\Lambda|^{1/2}}$$

Broader posteriors (larger determinant) are preferred for model selection.

3.2 Variational Inference and ELBO

The evidence lower bound (ELBO) decomposes the log marginal likelihood:

$$\mathcal{L}(w) = \mathbb{E}_{q(w)} \left[\log \frac{p(w, D)}{q(w)} \right] = \log P(D) - \text{KL}[q(w) \| p(w|D)]$$

Maximizing the ELBO minimizes the KL divergence to the true posterior. The gradient with respect to variational parameters λ is:

$$\nabla_{\lambda} \mathbb{E}_{q(w)} [\log p(w, D) - \log q_{\lambda}(w)] = \mathbb{E}_{q(w)} \left[\nabla_{\lambda} \log \frac{p(w, D)}{q_{\lambda}(w)} \right]$$

3.3 Reparametrization Trick

The reparametrization trick enables gradient estimation through sampling. With $\epsilon \sim \mathcal{N}(0, I)$ and $w = \mu + \Sigma^{1/2}\epsilon$:

$$\nabla_{\lambda} \mathbb{E}_{q(w)} \left[\log \frac{p(w, D)}{q_{\lambda}(w)} \right] = \mathbb{E}_{p(\epsilon)} \left[\nabla_{\lambda} \log \frac{p(\mu + \Sigma^{1/2}\epsilon, D)}{q_{\lambda}(\mu + \Sigma^{1/2}\epsilon)} \right]$$

This decouples the gradient from the expectation, allowing backpropagation through sampling.

3.4 Understanding KL Divergence Modes

Reverse KL (Mode-seeking): $\text{KL}[q \| P]$ penalizes placing probability mass where p is zero:

$$\mathbb{E}_{q(z)} \left[\log \frac{q(z)}{p(z|D)} \right] = \mathbb{E}_{q(z)} \log q(z) - \mathbb{E}_{q(z)} \log p(z|D)$$

When $p(z|D) \rightarrow 0$ but $q(z) > 0$, the cross-entropy diverges to $-\infty$, heavily suppressing wide tails.

Forward KL (Mass-covering): $\text{KL}[p||q]$ penalizes missing probability mass:

$$\mathbb{E}_{p(z|D)} \left[\log \frac{p(z|D)}{q(z)} \right] = \mathbb{E}_{p(z|D)} \log p(z|D) - \mathbb{E}_{p(z|D)} \log q(z)$$

When $p(z|D) > 0$ but $q(z) \approx 0$, it diverges to ∞ , forcing broad coverage.

3.5 Sequential Learning

In streaming or continual learning settings, the posterior from the previous batch becomes the prior for the next:

$$p(\theta|D_{1:t}) \propto p(D_t|\theta)p(\theta|D_{1:t-1})$$

This enables efficient online updates without reprocessing historical data.

4 Part IV: Deep Learning Architectures

4.1 Deep Feature Learning

Rather than relying on hand-crafted features, deep learning learns feature representations:

$$f(x) = w^T \tilde{\Phi}(x)$$

where both the weights w and the feature transformation $\tilde{\Phi}$ are learned jointly. The simplest deep model has a single hidden layer:

$$f(x) = w^T g(Ax + b)$$

with non-linear activation $g(\cdot)$.

4.2 Multi-layer Perceptrons

A multi-layer network stacks multiple transformations:

$$h_0 = x, \quad h_\ell = g(U_\ell h_{\ell-1} + b_\ell), \quad \ell = 1, \dots, L$$

where $g(\cdot)$ is typically ReLU: $g(z) = \max(0, z)$.

Loss functions align with probabilistic models:

- Squared error $\Leftrightarrow$ Gaussian likelihood
- Binary cross-entropy $\Leftrightarrow$ Bernoulli likelihood
- Absolute error $\Leftrightarrow$ Laplace likelihood

Regularization via L^1 or L^2 penalties with early stopping prevents overfitting.

4.3 Convolutional Neural Networks

Convolutions exploit two key properties of natural signals:

1. **Locality:** Features depend only on small neighborhoods
2. **Translation Invariance:** Patterns repeat across space

The 1D convolution is:

$$(f * g)(t) = \int f(a)g(t-a)da, \quad \text{or discretely: } (X * w)_i = \sum_{j=0}^{k-1} X_{i+j}w_j$$

The 2D case extends naturally:

$$(X * K)_{ij} = \sum_a \sum_b X_{i+a, j+b} K_{ab}$$

A typical convolutional block consists of:

- Convolutional stage: Affine transform with weight sharing
- Detector stage: Nonlinearity (e.g., ReLU)
- Pooling stage: Max or average pooling over spatial regions

4.4 Network Depth and Architecture Improvements

Residual Connections enable very deep networks by using skip connections:

$$h_\ell = h_{\ell-1} + M(h_{\ell-1})$$

This prevents vanishing gradients and enables information to flow across many layers.

Batch Normalization rescales activations to zero mean, unit variance per batch:

$$\mu_d = \frac{1}{M} \sum_i x_i^d, \quad \hat{x}_i^d = \frac{x_i^d - \mu_d}{\sqrt{\sigma_d^2 + \epsilon}}, \quad y_i^d = \gamma_d \hat{x}_i^d + \beta_d$$

Learnable scale and shift parameters γ_d, β_d restore expressiveness. BN reduces internal covariate shift and enables faster training.

4.5 Bayesian Approaches to Deep Models

Bayesian Last Layer: A practical middle ground between point estimates and full Bayesian inference:

$$p(y|x) = \int p(y|w^T \phi(x)) p(w|D) dw$$

Find point estimates for early layers and infer posterior over the final layer weights.

Deep Ensembles: Train S networks with different initializations and treat as an ensemble:

$$p(y|x) \approx \frac{1}{S} \sum_{s=1}^S p(y|x, \theta^{(s)})$$

Naturally captures multimodal posterior structures and weight permutation symmetries that challenge single-model Bayesian methods.

Bayesian Inference Challenges: Neural networks exhibit many symmetries (e.g., permutation of hidden units), creating disconnected modes in the posterior. Laplace approximation typically captures only one mode, while variational inference is often mode-seeking, and MCMC may mix poorly. Ensembles address this naturally.

5 Part V: Uncertainty Quantification and Probabilistic Models

5.1 Uncertainty in Neural Networks

Instead of point estimates θ^* , we can assign distributions $p(\theta|D)$ to capture parameter uncertainty. The posterior predictive distribution is:

$$p(y|x, D) = \int p(y|x, \theta)p(\theta|D)d\theta$$

For Gaussian likelihoods, this decomposes into epistemic (parameter) and aleatoric (data) uncertainty.

5.2 Laplace Approximation for Deep Networks

Find the MAP estimate: $\theta^* = \operatorname{argmax}_{\theta} \log p(\theta|D)$

Approximate the posterior with a Gaussian:

$$p(\theta|D) \approx \mathcal{N}(\theta|\theta^*, \mathbb{H}^{-1})$$

where the Hessian has the form:

$$\mathbb{H} = \alpha I + \beta \sum_i [\nabla_{\theta} f(x_i, \theta)][\nabla_{\theta} f(x_i, \theta)]^T$$

Predictive uncertainty combines epistemic and aleatoric components:

$$p(y|x, D) \approx \mathcal{N}(y|f_{\theta^*}(x), [\nabla f_{\theta^*}(x)]^T \mathbb{H}^{-1} [\nabla f_{\theta^*}(x)] + \sigma^2)$$

5.3 Model Calibration

A classifier is well-calibrated when predicted probabilities match empirical frequencies:

$$P(\text{true label} = k \mid \text{predicted prob} = p) \approx p$$

For evaluation, binning predictions into intervals $I_m = [c_{m-1}, c_m]$:

- Accuracy: $\text{acc}_m = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}[\hat{y}_i = y_i]$
- Confidence: $\text{conf}_m = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$

Expected Calibration Error (ECE) measures the gap:

$$\text{ECE} = \sum_m \frac{|B_m|}{N} |\text{acc}_m - \text{conf}_m|$$

Temperature Scaling adjusts predicted probabilities without changing decisions:

$$p_{\text{scaled}}(y = k|x) = \frac{\exp(z_k/T)}{\sum_j \exp(z_j/T)}$$

Find optimal T by maximizing likelihood on a validation set.

5.4 Epistemic vs. Aleatoric Uncertainty

Total predictive uncertainty decomposes into:

$$p(y|x, D) = \int p(y|x, \theta)p(\theta|D)d\theta$$

- **Epistemic uncertainty** (model/parameter uncertainty): Reduces with more data. Captures $\text{Var}_\theta[\mathbb{E}[y|\theta]]$
- **Aleatoric uncertainty** (data/measurement uncertainty): Irreducible. Captures $\mathbb{E}_\theta[\text{Var}(y|\theta)]$

For Gaussian models:

$$\text{Var}(y) = \text{Var}_\theta[\mu(\theta)] + \mathbb{E}_\theta[\sigma^2(\theta)]$$

5.5 Variational Autoencoders

VAEs define a generative model through latent variables z :

$$p_\theta(x) = \int p_\theta(x|z)p(z)dz, \quad p(z) = \mathcal{N}(0, I)$$

The ELBO decomposes into reconstructive and regularization terms:

$$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z|x)||p(z))}_{\text{Regularization}}$$

The encoder and decoder are typically parameterized as:

$$q_\phi(z|x) = \mathcal{N}(z|\mu_\phi(x), \text{diag}(\sigma_\phi(x))), \quad p_\theta(x|z) = \mathcal{N}(x|\mu_\theta(z), \text{diag}(\sigma_\theta(z)))$$

The reparametrization trick enables gradient-based optimization.

5.6 Semi-Supervised Learning with VAEs

Incorporating both labeled and unlabeled data:

Supervised loss (on labeled pairs):

$$\mathcal{L}(x_i, y_i) = \mathbb{E}_{q(z|x_i, y_i)}[\log p(x_i, y_i|z) - \log q(z|x_i, y_i)]$$

Unsupervised loss (on unlabeled):

$$\mathcal{U}(x_j) = \mathbb{E}_{q(y, z|x_j)}[\log p(x_j, y|z) - \log q(z, y|x_j)]$$

Combine with a classification term on labeled data to jointly optimize reconstruction and label prediction.

5.7 Neural Networks as Gaussian Processes

In the limit of infinite hidden-layer width with random initialization, a single-hidden-layer network:

$$f(x) = a + w^T g(Vx + b)$$

converges to a Gaussian process:

$$f \sim \mathcal{GP}(0, k(x, x'))$$

with kernel determined by the architecture:

$$k(x, x') = \sigma_a^2 + \sigma_w^2 \mathbb{E}_{b,v} [g(v^T x + b)g(v^T x' + b)]$$

5.8 Information Theory Basics

Mutual information measures dependence between random variables:

$$I(X; Y) = D_{\text{KL}}(p(x, y) \| p(x)p(y)) = H(X) - H(X|Y)$$

For Gaussians, entropy is:

$$H(X) = \frac{1}{2} \log \det(2\pi e \Sigma)$$

6 Part VI: Advanced Applications and Inference Methods

6.1 Active Learning

In active learning, we strategically select which unlabeled examples to label, maximizing model improvement per labeling cost:

- Labeled set: $\{(x_i, y_i)\}_{i=1}^n$
- Unlabeled pool: $\{x_j\}_{j=1}^m$ with $m \gg n$

Simple strategies query points where model predictions are least confident (high entropy). More sophisticated approaches use Bayesian methods to measure uncertainty over the parameters.

6.2 Bayesian Active Learning by Disagreement (BALD)

BALD selects points that maximize parameter uncertainty by leveraging ensemble disagreement:

$$x^* = \operatorname{argmax}_x I(y; \theta | D, x) = H(y | D, x) - \mathbb{E}_{\theta \sim p(\theta | D)}[H(y | x, \theta)]$$

The first term (average uncertainty) captures overall model uncertainty. The second term (expected conditional entropy) captures uncertainty that the model would retain after learning the label. The difference isolates epistemic (parameter) uncertainty.

6.3 Bayesian Optimization

For expensive black-box functions $f(x)$, Bayesian optimization enables efficient sequential evaluation:

1. Fit a probabilistic surrogate model $p(f | D)$ (typically Gaussian Process)
2. Select next evaluation point via acquisition function that balances exploration and exploitation
3. Evaluate f at selected point, update surrogate
4. Repeat

6.4 Acquisition Functions

Common acquisition functions trade off exploration and exploitation:

Expected Improvement (EI): Favors points likely to exceed current best:

$$\alpha_{\text{EI}}(x) = \mathbb{E}_f[\max(0, f(x) - f_{\text{best}})]$$

Upper Confidence Bound (UCB): Balances mean and uncertainty:

$$\alpha_{\text{UCB}}(x) = \mu(x) + \beta \sqrt{v(x)}$$

Entropy Search: Favors points that most reduce uncertainty about the optimum:

$$\alpha_{\text{ES}}(x) = H(x^* | D) - \mathbb{E}_{y|x}[H(x^* | D \cup \{(x, y)\})]$$

6.5 Langevin Dynamics for Posterior Sampling

Rather than approximation, we can sample directly from the posterior using Langevin dynamics:

$$\theta_{t+1} = \theta_t - \frac{\eta}{2} \nabla \log p(\theta_t | D) + \sqrt{\eta} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

With appropriate step size decay, this converges to samples from the posterior. The gradient term drives movement towards high-probability regions while noise enables exploration and prevents convergence to local modes.

6.6 Stochastic Gradient Langevin Dynamics (SGLD)

SGLD scales Langevin dynamics to large datasets using minibatches:

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{2} \nabla \tilde{L}(\theta_t) + \sqrt{\eta_t} \epsilon_t$$

where $\tilde{L}(\theta) = \frac{N}{M} \sum_{i \in \text{batch}} \ell_i(\theta)$ rescales the minibatch loss. The noise from two sources—the minibatch gradient noise and the added Gaussian-drive efficient posterior exploration. As step sizes decay appropriately ($\eta_t \rightarrow 0$), SGLD asymptotically samples from the posterior.

6.7 Wake-Sleep Algorithm

The wake-sleep algorithm alternates between two phases for training deep generative models:

Wake phase: Update decoder using real data and learned encoder:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathbb{E}_{q_{\phi}(z|x)} [-\log p_{\theta}(x|z)]$$

Improves model likelihood on real data.

Sleep phase: Update encoder using generated data from the decoder:

$$\phi \leftarrow \phi - \eta \nabla_{\phi} \mathbb{E}_{p_{\theta}(x,z)} [-\log q_{\phi}(z|x)]$$

Improves encoder agreement with the model’s implicit posterior.

This alternating approach avoids the KL divergence asymmetry issues present in standard VAE training and enables exploration of multimodal posterior structures. The algorithm naturally handles scenarios where a simple factorized Gaussian encoder would be insufficient.